Interfaces and Packages

Module 4

Abstract Classes

In Java, an abstract class is a class that cannot be instantiated and is designed to be extended by other classes. It is used to achieve partial abstraction, where some methods are implemented while others are left for subclasses to define. An abstract class is declared using the abstract keyword. It may contain:

- Abstract methods (methods without a body)
- Concrete methods (methods with implementation)
- Constructors
- Instance variables
- Static and final methods

Abstract Classes

```java
// Abstract class
abstract class Animal {
  // Abstract method (does not have a body)
  public abstract void animalSound();

  public void sleep() {
    System.out.println("Zzz");
  }
}

// Subclass (inherit from Animal)
class Pig extends Animal {
  public void animalSound() {
    System.out.println("The pig says: wee wee");
  }
}

class Main {
  public static void main(String[] args) {
    Pig myPig = new Pig(); // Create a Pig object
    myPig.animalSound();
    myPig.sleep();
  }
}
```

Abstract Classes – Abstract vs Concrete methods

```
abstract class DemoAbstractClass {
    abstract void display();
}

public class JavaApplication {
    public static void main(String[] args)
    {
        DemoAbstractClass AC = new DemoAbstractClass() {
            void display()
            {
                System.out.println("Hi.");
            }
        };
        AC.display();
        System.out.println("How are you?");
    }
}
```

```
abstract class DemoAbstractClass {
    abstract void display();
}

class ConcreteClass extends DemoAbstractClass {
    void display()
    {
        System.out.println("Hi.");
    }
}

public class JavaApplication {
    public static void main(String[] args)
    {
        ConcreteClass C = new ConcreteClass();
        C.display();
        System.out.println("How are you?");
    }
}
```

Abstract Classes – Abstract vs Concrete methods

Abstract Class	Concrete Class
An abstract class is declared using abstract modifier.	A concrete class is not declared using abstract modifier.
An abstract class cannot be directly instantiated using the new keyword.	A concrete class can be directly instantiated using the new keyword.
An abstract class may or may not contain abstract methods.	A concrete class cannot contain an abstract method.
An abstract class cannot be declared as final.	A concrete class can be declared as final.
Implement an interface is possible by not providing implementations of all of the interface's methods. For this a child class is needed..	Easy implementation of all of the methods in the interface.

Abstract Classes – Use cases and Design Patterns

Abstract classes serve as incomplete blueprints in object-oriented programming, used to define a common foundation for related classes while providing shared state and partial implementations. Unlike interfaces, which represent a "can-do" contract, abstract classes represent an "is-a" relationship within a clear inheritance hierarchy QuickStart .

Common Use Cases

Abstract classes are ideal when you have multiple classes that share significant logic but also require specialized behavior .

- **Base Frameworks & APIs:** Developers use them to create structural guides for future extensions. For example, a BaseAPI class might handle common HTTP setup and interceptors, while subclasses define specific endpoint logic .

- **Database Connections:** A DatabaseConnection abstract class can store shared credentials and port information, leaving specific driver implementations (like MySQL or Oracle) to subclasses .

- **Game Development:** An abstract GameObject can provide standard update() and render() methods that every entity in a game must follow .

- **State Maintenance:** Because they can contain member variables (fields), they are used when subclasses need to inherit and modify shared data, such as a Bank Account base class tracking a balance

Abstract Classes – Use cases and Design Patterns

Key Design Patterns

Several classic design patterns rely on the structure of abstract classes to decouple code and enforce specific workflows.

- **Template Method Pattern:** This is the most common use case. An abstract class defines the "skeleton" of an algorithm, providing default steps but leaving specific "hooks" as abstract methods for subclasses to implement .
  Example: A File Processor that reads and logs data the same way but parses CSV vs. JSON differently .

- **Abstract Factory Pattern:** Often described as a "factory of factories," it provides an interface for creating families of related or dependent objects (e.g., UI components for different OSs) without specifying their concrete classes .

- **Factory Method Pattern:** Uses an abstract class to define a creation method. Subclasses then override this method to change the type of object that will be created, allowing for flexible object creation .

- **Strategy Pattern:** While often implemented with interfaces, abstract classes are used when the interchangeable behaviors share significant common logic

Interfaces

An interface in Java is a blueprint that defines a set of methods a class must implement without providing full implementation details. It helps achieve abstraction by focusing on what a class should do rather than how it does it. Interfaces also support multiple inheritance in Java.

- A class must implement all abstract methods of an interface.
- All variables in an interface are public, static, and final by default.
- Interfaces can have default, static, and private methods.

```java
import java.io.*;
// Interface Declared
interface testInterface {
    // public, static and final
    final int a = 10;
    // public and abstract
    void display();
}

// Class implementing interface
class TestClass implements testInterface {

    // Implementing the capabilities of Interface
    public void display(){
      System.out.println("Geek");
    }
}

class Geeks{

    public static void main(String[] args){

        TestClass t = new TestClass();
        t.display();
        System.out.println(t.a);
    }
}
```

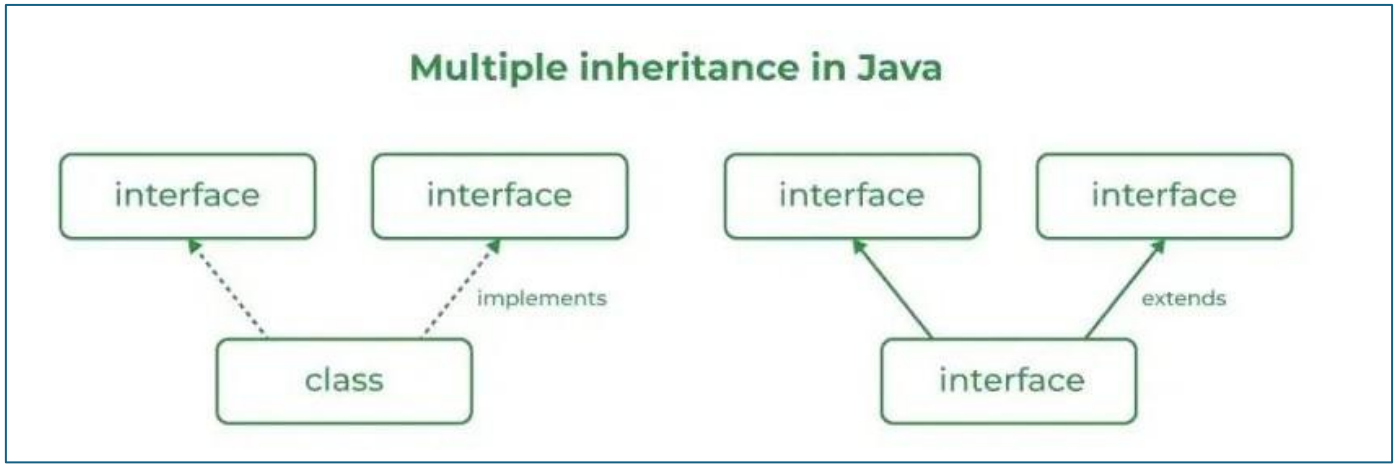
Interfaces – Defining and Implementing Interfaces

Key Aspects of Defining and Implementing Interfaces

- **Definition:** Defined using the interface keyword instead of class. All fields are public, static, and final (constants), and methods are implicitly public and abstract by default.
- **Implementation:** A class uses the implements keyword to conform to the interface. The class must provide concrete bodies for all methods defined in the interface.
- **Multiple Interfaces:** Unlike class inheritance, a class can implement multiple interfaces, separated by commas, allowing for flexible, multi-faceted design.
- **Default/Static Methods:** Modern Java (8+) allows interfaces to have default methods with implementations, allowing interface evolution without breaking existing code.
- **Why Use Them:** Interfaces are essential for achieving abstraction, loose coupling, and polymorphism, allowing different objects to be treated as the same type based on their capabilities, rather than their class hierarchy.

Interfaces – Multiple Inheritance with Interfaces

Java does not support multiple inheritance with classes to avoid ambiguity, but it supports multiple inheritance using interfaces.



Interfaces – Multiple Inheritance with Interfaces

```
import java.io.*;
// Add interface
interface Add{
    int add(int a,int b);
}
// Sub interface
interface Sub{
    int sub(int a,int b);
}

// Calculator class implementing Add and Sub
class Cal implements Add , Sub
{
    // Method to add two numbers
    public int add(int a,int b){
        return a+b;
    }

    // Method to sub two numbers
    public int sub(int a,int b){
        return a-b;
    }
}
```

```
class GFG{
    // Main Method
    public static void main (String[] args){

        // instance of Cal class
        Cal x = new Cal();
        System.out.println("Addition : " + x.add(2,1));
        System.out.println("Substraction : " + x.sub(2,1));
    }
}
```

Interfaces – Default and Static Methods

Default Methods

- Interfaces can define methods with default implementations.
- Useful for adding new methods to interfaces without breaking existing implementations.

```
interface TestInterface
{
    final int a = 10;

    default void display() {
        System.out.println("hello");
    }
}

// A class that implements the interface.
class TestClass implements TestInterface
{
    // Driver Code
    public static void main (String[] args) {
        TestClass t = new TestClass();
        t.display();
    }
}
```

Interfaces – Default and Static Methods

Static Methods

- Interfaces can now include static methods.
- These methods are called directly using the interface name and are not inherited by implementing classes.

```
interface TestInterface
{
    final int a = 10;
    static void display()
    {
        System.out.println("hello");
    }
}

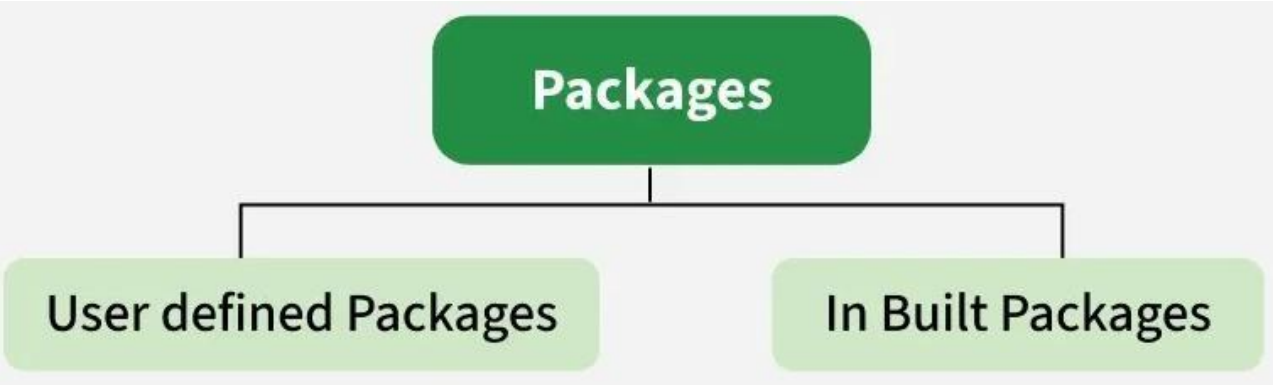
// A class that implements the interface.
class TestClass implements TestInterface
{
    // Driver Code
    public static void main (String[] args)
    {
        TestInterface.display();
    }
}
```

Packages

A package in Java is a mechanism to group related classes, interfaces, and sub-packages into a single unit. Packages help organize large applications, avoid naming conflicts, provide access protection, and make code modular and maintainable.

- Avoiding name conflicts (two classes with the same name can exist in different packages)
- Providing access control using public, protected, and default access
- Reusability: packaged code can be imported and used anywhere
- Encouraging modular programming

Types of Java Packages:



Packages - Built-in packages (java.util, java.io)

1. Built-in Packages

Built-in Packages comprise a large number of classes that are part of the Java API. Some of the commonly used built-in packages are:

- **java.lang:** Contains language support classes(e.g, classes that define primitive data types, math operations). This package is automatically imported.
- **java.io:** Contains classes for supporting input/output operations.
- **java.util:** Contains utility classes that implement data structures such as Linked Lists and Dictionaries, as well as support for date and time operations.
- **java.applet:** Contains classes for creating Applets.
- **java.awt:** Contains classes for implementing the components for graphical user interfaces (like buttons, menus, etc).

```java
import java.util.Random;   // built-in package

public class GFG{

    public static void main(String[] args) {

        // using Random class
        Random rand = new Random();

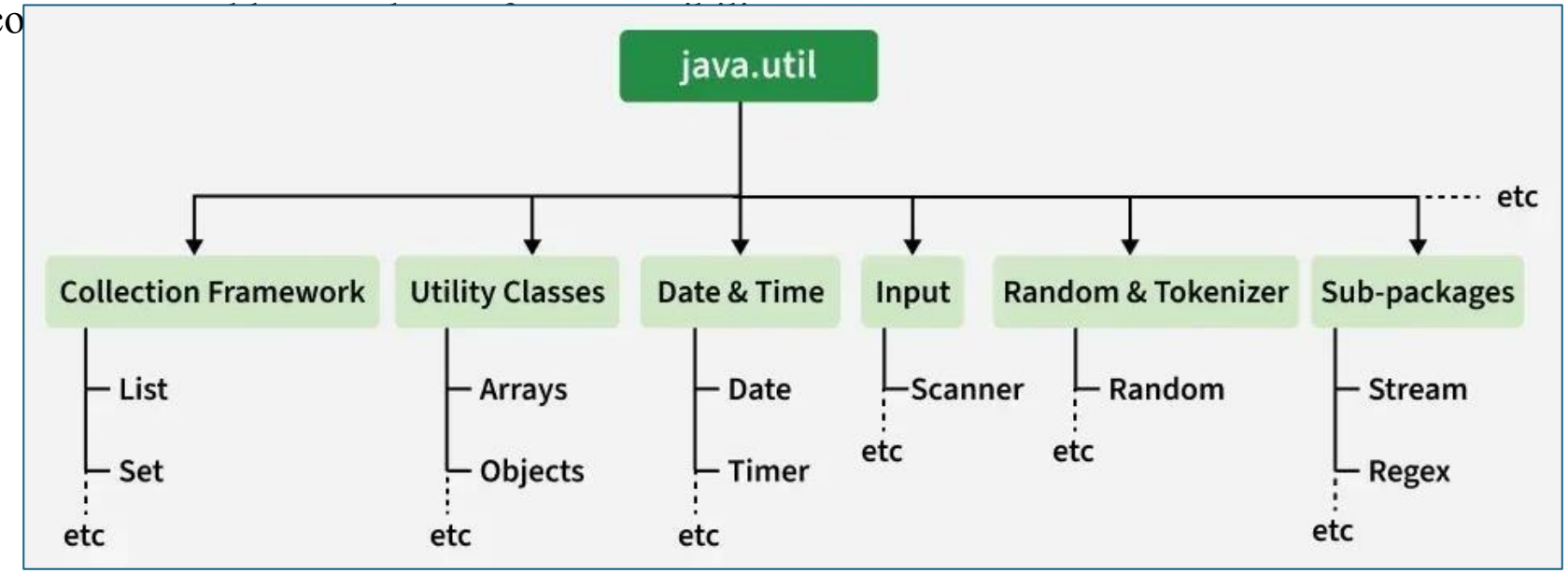
        // generates a number between 0-99
        int number = rand.nextInt(100);

        System.out.println("Random number: " + number);
    }
}
```

Packages - Built-in packages (java.util, java.io)

The java.util package is one of the most widely used packages in the Java programming language. It provides a set of utility classes that support data structures, date and time manipulation, random number generation, event handling, and other commonly used functionalities in Java programs.

- Provides a rich Collection Framework (List, Set, Map) for data handling.
- Includes utility classes (Collections, Arrays) for sorting and searching.
- Offers Date, Calendar, and Random for time and random operations.
- Supports Properties for configuration management.
- Contains co



Packages - Built-in packages (java.util, java.io)

Java IO Package is a crucial part of Java's core libraries that provides a comprehensive set of classes for input and output operations. These operations allow you to read from and write to a variety of data sources such as files, memory, and the console. The Java IO Package simplifies how data is processed in Java applications.

Core Components of Java IO Package

Java IO Package consists of various classes and interfaces that can be broadly categorized as follows:

- **Input Streams:** Used to read data from sources.
- **Output Streams:** Used for writing data to destinations.
- **Readers and Writers:** A specialized version of input and output streams for character data.

Packages – User-defined packages

2. User-defined Packages

- User-defined Packages are the packages that are defined by the user.

```java
package com.myapp;

public class Helper {
    public static void show() {
        System.out.println("Hello from Helper!");
    }
}
```

```java
import com.myapp.Helper;

public class Test {
    public static void main(String[] args) {
        Helper.show();
    }
}
```

Save the file as Helper.java, and compile it
javac Helper.java

Then Compile the Package:
javac –d . Helper.java

That package can be used in other class like this

Packages – Access control and Import statements

In Java, packages serve as containers for classes, providing both namespace management and access control.

1. Packages and Access Control

Packages determine the visibility of classes and their members through Access Modifiers:

- **Public:** The class or member is visible to all other classes, regardless of which package they are in.
- **Default (No keyword):** Also called **package-private**. The class or member is visible *only* to other classes within the same package.
- **Protected:** The member is visible to classes in the same package and to subclasses in different packages.
- **Private:** The member is visible only within the class where it is defined; packages do not grant additional access to private members.

Packages – Access control and Import statements

2. Import Statements

To use a class located in a different package, you must either use its Fully Qualified Name (e.g.,) or an statement.

- **Syntax:** *import package.name.ClassName;* **or** *import package.name.*;*
- **Specific Import:** Imports only a single class (e.g., import java.util.ArrayList;).
- **Wildcard Import (*):** Imports all public classes and interfaces within a package but **does not** import sub-packages (e.g., import java.util.*; does not import java.util.concurrent).
- **Static Import:** Allows access to static members (fields/methods) of a class without using the class name (e.g.,).

Key Implementation Rules

- **Placement:** The package declaration must be the first statement in the file. The import statements must follow the package declaration and appear before any class definitions.
- **Automatic Import:** The java.lang package (containing classes like and) and the current package are always imported implicitly.
- **Directory Structure:** The package name must match the physical directory structure on the disk.

Wrapper Classes

- In Java, wrapper classes allow primitive data types to be represented as objects. This enables primitives to be used in object-oriented features such as collections, generics, and APIs that require objects.
- Each wrapper class encapsulates a corresponding primitive value inside an object (e.g., Integer for int, Double for double).
- Java provides wrapper classes for all eight primitive data types to support object-based operations.

Primitive Data Type	Wrapper Class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

Why Wrapper Classes Are Needed

- Wrapper classes are required in Java for the following reasons:
- Java collections (ArrayList, HashMap, etc.) store only objects, not primitives.
- Wrapper objects allow primitives to be used in object-oriented features like methods, synchronization, and serialization.
- Objects support null values, while primitives do not.
- Wrapper classes provide utility methods such as compareTo(), equals(), and toString()

Wrapper Classes – Autoboxing and Unboxing

1. Autoboxing

- The automatic conversion of primitive types to the object of their corresponding wrapper classes is known as autoboxing. For example: conversion of int to Integer, long to Long, double to Double, etc.

Java program to demonstrate the automatic conversion of primitive to wrapper class (Autoboxing).

```
import java.util.ArrayList;

class Geeks {
    public static void main(String[] args) {
        char ch = 'a';

        // Autoboxing: char -> Character
        Character c = ch;

        ArrayList<Integer> list = new ArrayList<>();
        // Autoboxing: int -> Integer
        list.add(25);
        System.out.println(list.get(0));
    }
}
```

Wrapper Classes – Autoboxing and Unboxing

2. Unboxing

- Unboxing is the automatic conversion of a wrapper class object back into its corresponding primitive type.

```
import java.util.ArrayList;

class Geeks {
    public static void main(String[] args) {

        Character ch = 'a';
        // Unboxing: Character -> char
        char c = ch;

        ArrayList<Integer> list = new ArrayList<>();
        list.add(24);
        // Unboxing: Integer -> int
        int num = list.get(0);

        System.out.println(num);
    }
}
```

Wrapper Classes – Converting between primitives and objects

Java Wrapper classes provide a mechanism to wrap primitive data types (like int, char, double) into objects. This conversion is essential for using primitives in object-oriented features such as the Java Collections Framework (e.g., ArrayList, HashMap) or with Generics.

Core Conversion Concepts

Conversion between primitives and objects occurs through two primary processes:

- **Autoboxing (Primitive to Object):** The automatic conversion performed by the Java compiler.
  - *Explicit Method:* Integer.valueOf(primitiveValue).
  - *Automatic Example:* Integer obj = 10;.
- **Unboxing (Object to Primitive):** The automatic conversion of a wrapper object back into its corresponding primitive type.
  - *Explicit Method:* obj.intValue(), obj.doubleValue(), etc.
  - *Automatic Example:* int num = obj;

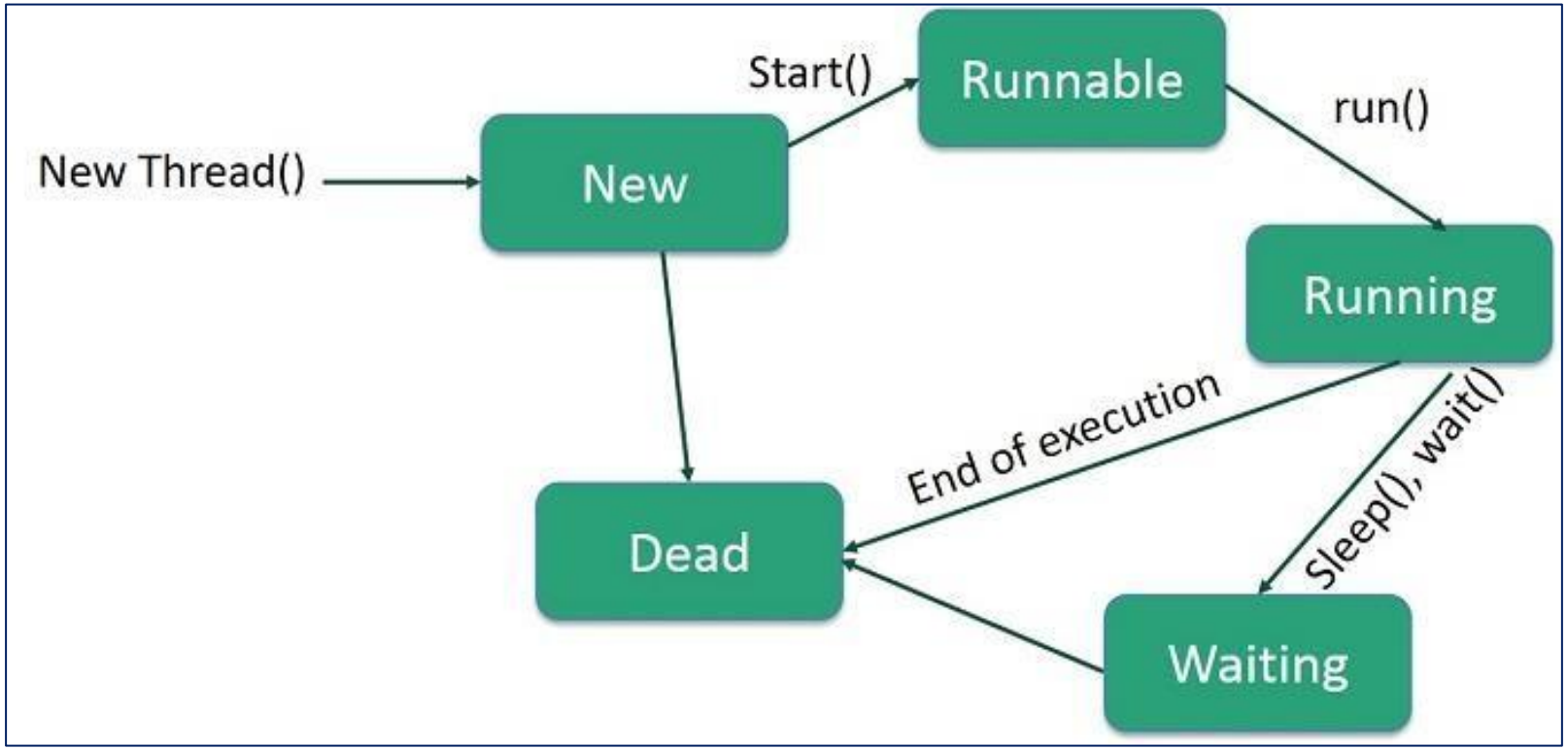
Multithreading

- Java is a *multi-threaded programming language* which means we can develop multi-threaded program using Java. A multi-threaded program contains two or more parts that can run concurrently and each part can handle a different task at the same time making optimal use of the available resources specially when your computer has multiple **CPUs**.
- By definition, multitasking is when multiple processes share common processing resources such as a CPU. Multi-threading extends the idea of multitasking into applications where you can subdivide specific operations within a single application into individual threads. Each of the threads can run in parallel. The OS divides processing time not only among different applications, but also among each thread within an application.

Java Multithreading

- Multi-threading enables you to write in a way where multiple activities can proceed concurrently in the same program. To achieve the multithreading (or, write multithreaded code), you need **java.lang.Thread class**.

Multithreading – thread lifecycle



Multithreading – thread lifecycle

Following are the stages of the life cycle −

- **New** − A new thread begins its life cycle in the new state. It remains in this state until the program starts the thread. It is also referred to as a **born thread**.

- **Runnable** − After a newly born thread is started, the thread becomes runnable. A thread in this state is considered to be executing its task.

- **Waiting** − Sometimes, a thread transitions to the waiting state while the thread waits for another thread to perform a task. A thread transitions back to the runnable state only when another thread signals the waiting thread to continue executing.

- **Timed Waiting** − A runnable thread can enter the timed waiting state for a specified interval of time. A thread in this state transitions back to the runnable state when that time interval expires or when the event it is waiting for occurs.

- **Terminated (Dead)** − A runnable thread enters the terminated state when it completes its task or otherwise terminates.

Multithreading – thread creation : extending thread and runnable

Java's multithreading system is built upon the Thread class, its methods, and its companion interface, Runnable.
To create a new thread, your program will either **extend Thread** or **implement the Runnableinterface.**

Method 1: Extending Thread Class

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread is running...");
    }

    public static void main(String[] args) {
        MyThread t1 = new MyThread(); // thread object
        t1.start(); // starts thread
    }
}
```

- run() → contains the task
- start() → creates a new thread and calls run() internally
- Never call run() directly → it won't create a new thread.

Multithreading – thread creation : extending thread and runnable

Method 2: Implementing Runnable Interface

```
class MyRunnable implements Runnable {
    public void run() {
        System.out.println("Thread using Runnable...");
    }

    public static void main(String[] args) {
        MyRunnable obj = new MyRunnable();
        Thread t1 = new Thread(obj);
        t1.start();
    }
}
```

- Runnable separates **task from thread**
- Better design (used in real projects)

Multithreading – thread creation : extending thread and runnable

Multiple Threads Example

```
class MyThread extends Thread {
    public void run() {
        for(int i = 1; i <= 5; i++) {
            System.out.println(Thread.currentThread().getName() + " - " + i);
        }
    }

    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        MyThread t2 = new MyThread();

        t1.start();
        t2.start();
    }
}
```

Multithreading – Synchronization and Inter-thread communication

Synchronization in Java

- **Synchronization** is a mechanism used to **control access to shared resources** in a multithreaded environment. It ensures that **only one thread executes a critical section at a time**, preventing data inconsistency.

Need for Synchronization

- When multiple threads access and modify shared data simultaneously, it may lead to:
- Race conditions
- Data inconsistency
- Unexpected results

Key Concept: Critical Section

- A **critical section** is a part of the program where shared resources are accessed.

Multithreading – Synchronization and Inter-thread communication

Types of Synchronization

1. **Method-Level Synchronization:** A method is declared as synchronized, allowing only one thread to execute it at a time.
2. **Block-Level Synchronization:** Only a specific block of code is synchronized instead of the entire method, improving efficiency.
3. **Static Synchronization:** Used for static methods where the **class-level lock** is applied instead of object-level lock.

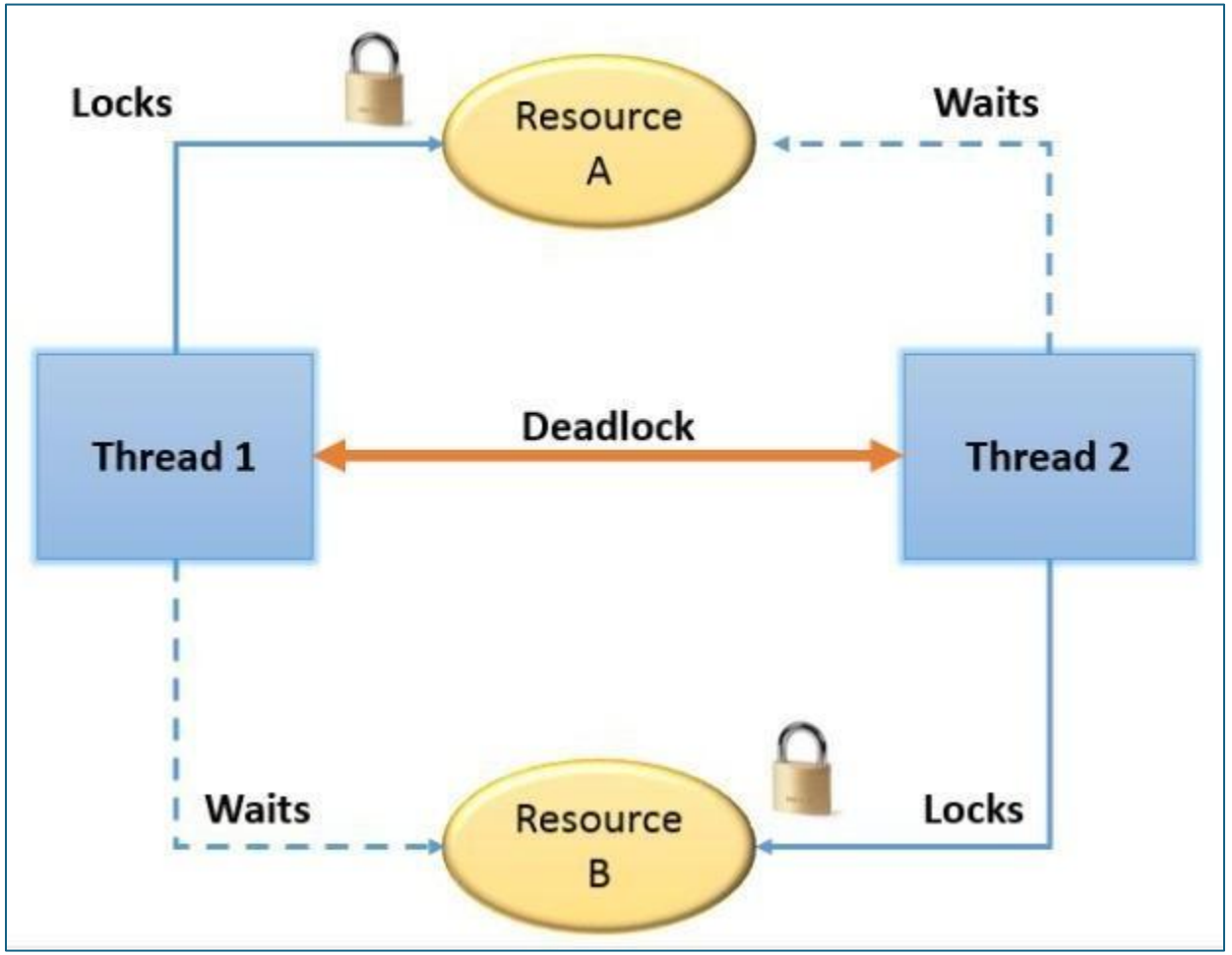
Lock Mechanism

- Every object in Java has an **intrinsic lock (monitor)**
- When a thread enters a synchronized block/method, it acquires the lock
- Other threads must wait until the lock is released

Problems in Synchronization

- **Deadlock:** Deadlock occurs when two or more threads are waiting indefinitely for each other's resources.
- **Starvation:** A thread may not get CPU resources for a long time due to priority issues.

Multithreading – Synchronization and Inter-thread communication



Multithreading – Synchronization and Inter-thread communication

Inter-thread communication is a mechanism that allows threads to **communicate and coordinate their execution** instead of competing for resources. It is mainly used in scenarios like:

- Producer–Consumer problem
- Resource sharing systems

Methods Used

Method	Description
wait()	Causes the thread to wait until notified
notify()	Wakes up one waiting thread
notifyAll()	Wakes up all waiting threads

Working Mechanism

- A thread calls wait() → releases lock and enters waiting state
- Another thread calls notify() or notifyAll() → wakes waiting thread
- The awakened thread competes again to acquire the lock

Multithreading – Synchronization and Inter-thread communication

Key Rules

- These methods must be used inside a synchronized block/method
- They are defined in the Object class
- Always use while condition to avoid logical errors

Difference: Synchronization vs Inter-Thread Communication

Feature	Synchronization	Inter-Thread Communication
Purpose	Control access	Coordinate execution
Focus	Mutual exclusion	Cooperation
Mechanism	Locks	wait/notify
Example	Prevent race condition	Producer–Consumer

Thank
You!